Systèmes Embarqués Communicants

Spécification et modélisation de programme

TP Noté - Aventurier du rail

Auteur :
M. Thomas METAIS
M. Guillaume PIRAUBE
M. Jean-Marc KERVIL
M. Aymeric ZIMINSKI

Version du
19 mars 2026

Table des matières

1 Introduction **2**

2 Conception des classes **3**
2.1 Classe Attaque 3
2.2 Classe Animal 3
2.3 Classes dérivées 4
2.3.1 Lion 4
2.3.2 Loup 4
2.3.3 Ours 4
2.3.4 Pierre 4

3 Implémentation **4**

4 Résultats et tests **5**

5 Conclusion **5**

1 Introduction

Dans ce TP, nous implémentons un mini-jeu inspiré du jeu de la vie, où différents animaux évoluent sur une grille. Chaque animal a des comportements spécifiques : type d'attaque (pierre, feuille, ciseaux), et mode de déplacement. Le jeu se déroule par tours : les animaux se déplacent, puis combattent s'ils se trouvent sur la même case. Le jeu s'arrête lorsque seuls les animaux immobiles (Pierres) restent en vie.

Les animaux implémentés sont :

— **Lion** : se déplace en diagonale, attaque avec feuille ou ciseaux.
— **Loup** : se téléporte aléatoirement, attaque avec n'importe quel type.
— **Ours** : se déplace dans 8 directions possibles, attaque toujours avec feuille.
— **Pierre** : immobile, attaque toujours avec pierre.

Le système d'attaque suit les règles de pierre-feuille-ciseaux : pierre bat ciseaux, ciseaux bat feuille, feuille bat pierre. En cas d'égalité, l'attaquant perd.

2 Conception des classes

2.1 Classe Attaque

La classe `Attaque` représente le système d'attaque basé sur pierre-feuille-ciseaux. Elle contient un entier `type` (0 : pierre, 1 : feuille, 2 : ciseaux) et une méthode `resoudreAttaque` qui détermine le gagnant.

```
class Attaque {
private:
    int type; // 0: pierre, 1: feuille, 2: ciseaux
public:
    Attaque();
    Attaque(int a);
    bool resoudreAttaque(const Attaque &a) const;
    const string getNomAttaque();
};
```

2.2 Classe Animal

La classe `Animal` est abstraite et sert de base pour tous les animaux. Elle définit les attributs communs : nom, position (x,y), état vivant, et type d'attaque.

```
class Animal {
protected:
    string nom;
    int x, y;
    bool vivant;
    Attaque typeAttaque;
public:
    Animal(int maxX, int maxY);
    Animal(int maxX, int maxY, int a, int b);
    const string getNom();
    const int getX();
    const int getY();
    const bool getVivant();
    const Attaque getAttaque();
    void setVivant(bool v);
    bool attaque(Animal &a);
    virtual void setAttaque(Attaque atq) = 0;
```

```
    virtual void setAttaque() = 0;
    virtual void deplace(int maxX, int maxY) = 0;
};
```

2.3 Classes dérivées

Chaque animal hérite de `Animal` et implémente ses propres méthodes `setAttaque` et `deplace`.

2.3.1 Lion

Le Lion choisit aléatoirement entre feuille et ciseaux pour attaquer, et se déplace en diagonale (une case dans chaque direction).

2.3.2 Loup

Le Loup peut attaquer avec n'importe quel type et se téléporte à une position aléatoire sur la grille.

2.3.3 Ours

L'Ours attaque toujours avec feuille et se déplace dans une des 8 directions avec des pas de 1 ou 2 cases.

2.3.4 Pierre

La Pierre attaque toujours avec pierre et ne se déplace pas.

3 Implémentation

Le programme principal utilise un `vector` de `unique_ptr<Animal>` pour gérer les animaux de manière polymorphique. La grille est de taille 10x10, avec initialement 25% des cases occupées par des animaux (alternant Lion, Loup, Ours, Pierre).

La boucle principale effectue les étapes suivantes à chaque tour :

1. Déplace chaque animal vivant (sauf les Pierres qui sont immobiles).
2. Pour chaque animal déplacé, vérifie s'il y a collision avec d'autres animaux sur la même case.

3. En cas de collision, les animaux concernés choisissent une attaque et combattent selon les règles de pierre-feuille-ciseaux.
4. Le perdant du combat est marqué comme mort.
5. Affiche l'état actuel de la grille.
6. Pause d'une seconde pour observer l'évolution.
7. Répète jusqu'à ce que seuls les animaux immobiles (Pierres) soient encore vivants.

4 Résultats et tests

Nous avons compilé et exécuté le programme avec succès. Voici un exemple d'affichage de la grille au cours d'une partie :

```
|----|----|----|----|----|----|----|----|----|----|
| L  |    | W  |    | O  |    | P  |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
...
```

Le jeu évolue dynamiquement : les animaux se déplacent, entrent en collision et combattent. Les Lions se déplacent en diagonale, les Loups se téléportent, les Ours avancent de manière variée, tandis que les Pierres restent fixes. Les combats résolvent les conflits selon le système pierre-feuille-ciseaux, éliminant les perdants.

5 Conclusion

Ce TP nous a permis de mettre en pratique les concepts avancés de la programmation orientée objet en C++ : héritage, polymorphisme, classes abstraites et gestion dynamique de la mémoire avec `unique_ptr`. L'ajout du système d'attaque stratégique transforme le jeu de la vie classique en une simulation plus interactive et imprévisible.